Deliverable II: System Modeling & Architectural Design

Due: October 19, 2025

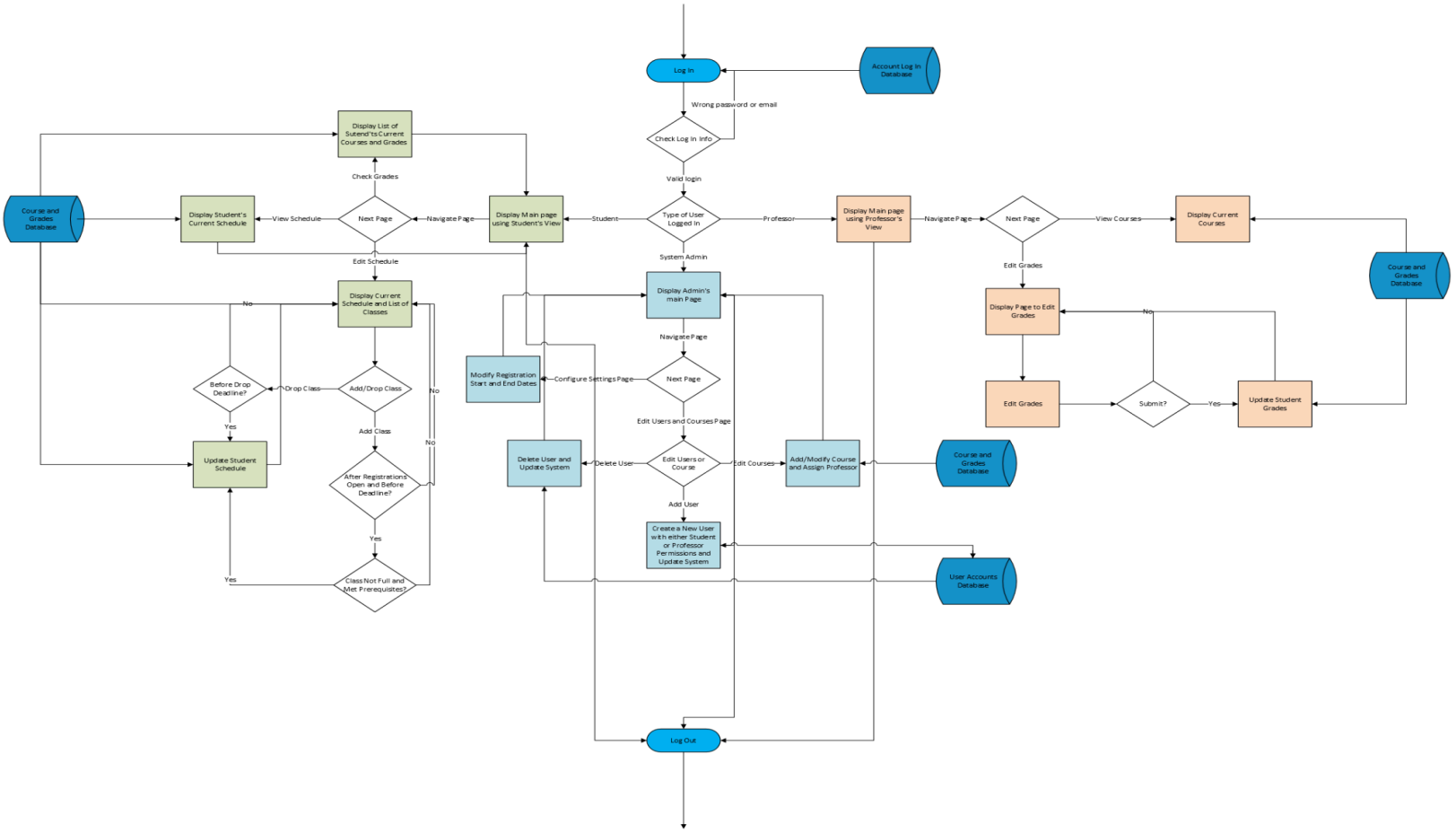
In this deliverable, your team will move from requirements to design. **The goal is to model your system's structure and behavior, and to capture early architectural decisions.** *This should demonstrate that your team can connect the "what" (requirements) with the "how" (design).*

1. Process Model (Activity Diagram)

- Draw an **activity diagram** for your entire system that shows the main flow of actions.
    - Include **all major roles** (users, external systems, subsystems).
    - Clearly label decisions (e.g., approve vs reject) and object flows (e.g., Request, Report, Payment).
    - Highlight parallel or alternative flows if relevant.
    - Write a short explanation (1-2 paragraphs) of why this workflow captures the essence of your system and how you structured it.

Answer:

Our registration system consists of three main users, which include students, professors, and system administrators. This activity diagram shows the general flow for each user, with the light green showing the flow for a student, the light blue showing the flow for the system admins, and the light orange showing the flow for professors. The flow captures the essence of our system by showing the different paths that users will take navigating through the system. The diagram starts with the system asking users to log in and then directing different users to their own subdiagram/subsystem. Each subsystem shows different tasks that each user can do, which were identified in our system's functional requirements. The diagram shows most of the main functional requirements of our system, like students being able to view and change their schedule, professors being able to control their courses and submit grades, and system admins being able to control system settings and users. The diagram also shows how certain actions require access to external components, such as the different databases highlighted in dark blue.



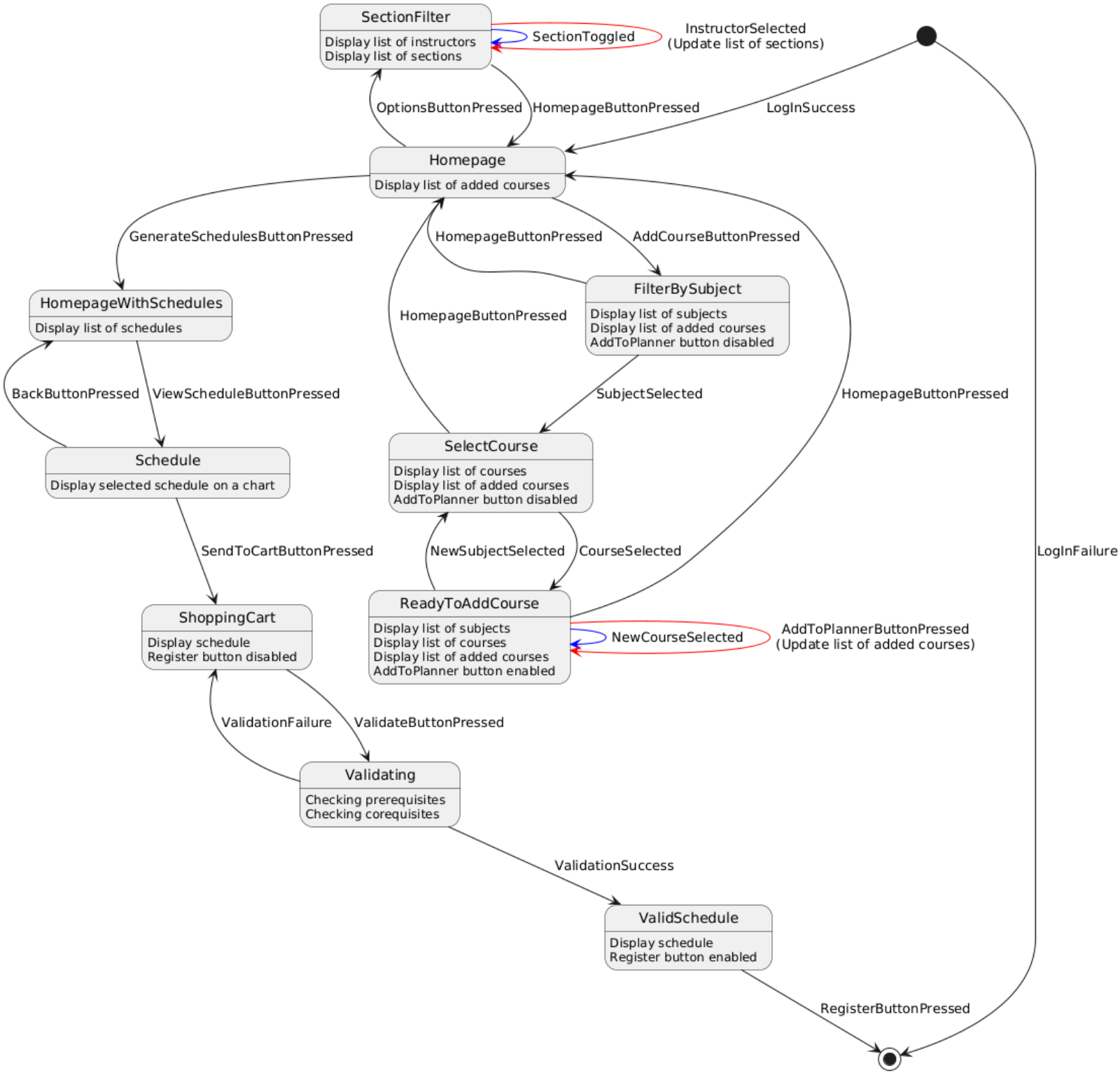
2. Behavioral Model

- Choose one of the following to illustrate dynamic behavior in your system:
  - **Sequence Diagram:** Show interactions among at least four participants (actors, components, services). Include at least one alternative path (e.g., error, rejection, failure).
  - **State Machine Diagram:** Model at least five states for one core component of your system. Show realistic events that cause transitions.
- Provide a brief description (4-5 sentences) explaining your assumptions and why you modeled the system behavior in this way.

Answer:

I chose a state machine diagram because it provides a better intuitive understanding than a sequence diagram would. The homepage has a button to add courses, and the menu to add courses requires the user to select a subject to filter by before displaying the list of courses. A button allows the user to add the course they chose to their planner, and doing so will keep the subject filter and the selected course the same, but trying to add a course that has already been added to the planner will do nothing. The

homepage will not display a list of schedules before the button to generate schedules is clicked. The schedule generator will make a list of every permutation of the enabled sections while ensuring there are no time conflicts.

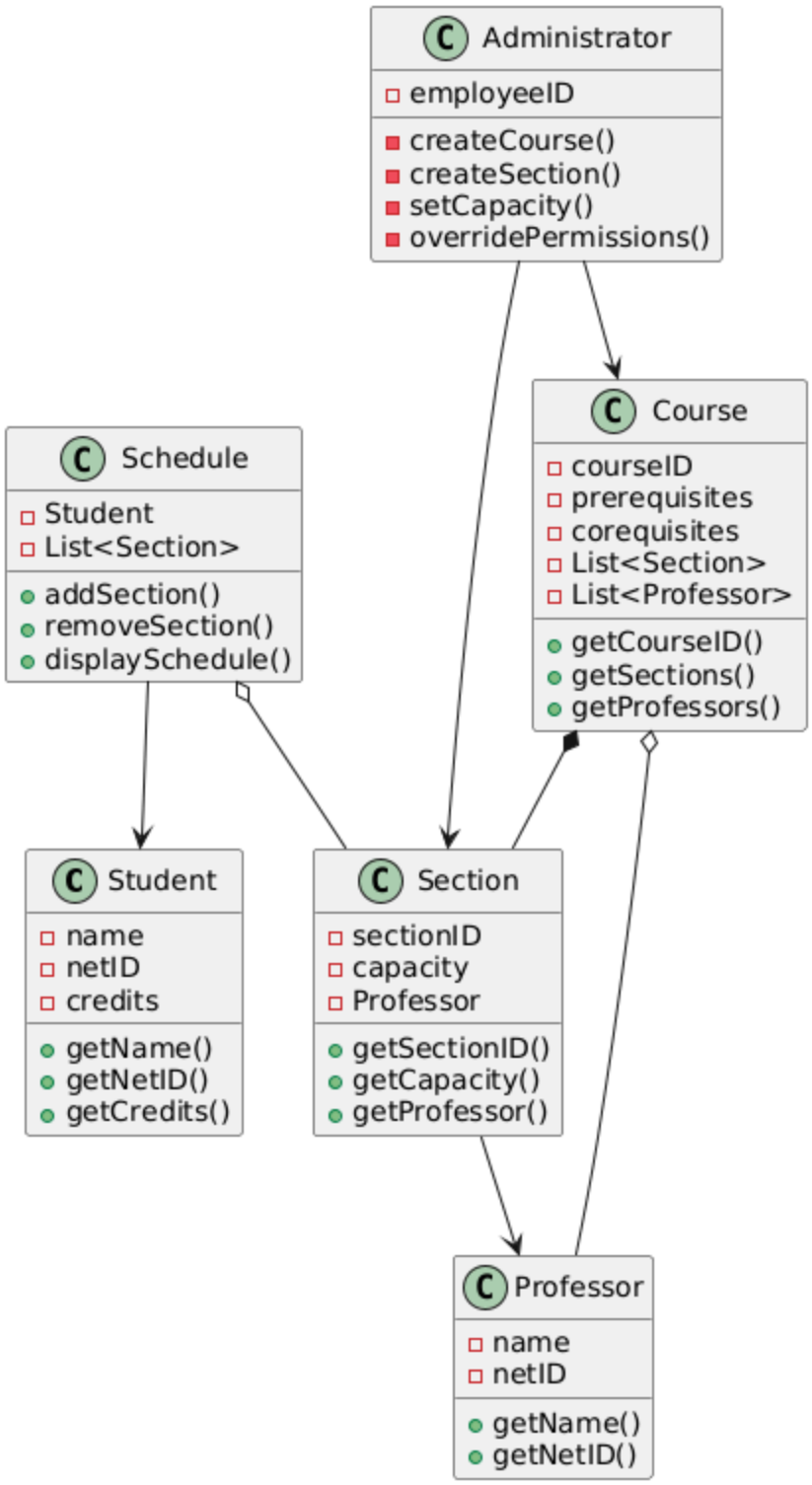


3. Structural Model (Class Diagram)

- Draw a **class diagram** that captures the structure of your system.
    - Include at least five classes with attributes and methods.
    - Show key relationships: generalization, aggregation/composition, and associations.
    - Provide a short explanation (3-5 sentences) of how you divided responsibilities among classes and why.

Answer:

The system has 3 main actors/users, which are the students, professors, and administrators, who we have made into classes. Each class has its own set of permissions and functions that the user can perform. For example, the Student class will have its own attributes for their grades and courses, and have the ability to add/drop courses. However, students are not able to change their grades, which is why we gave this responsibility to the Professor class. We also decided to make classes such as the Course and Schedule class. These classes will mainly have fewer responsibilities/functions, as their purpose is to create a general class for all courses or schedules, since all courses and schedules behave the same way and have the same information.



4. Architectural Design Decisions

- Select two of the non-functional requirements from Deliverable I and describe architectural design decisions that support them.

- State the requirement (e.g., performance, security, reliability).
- Explain the design decision (e.g., add caching, introduce encryption, replicate services).
- Provide justification (2-3 sentences) for how this decision impacts the overall system.

Answer:

- The first non-functional requirement chosen from Deliverable I is “The system shall at least be readily available to students at all hours from their designated registration time to the final day to drop classes 99% of the time”.
  - This requirement is reliability-based, focusing on the availability of the system, using uptime as the metric to evaluate this.
  - To maximize the amount of time the system is online, it can have multiple servers hosting the website to add a layer of redundancy. We can also incorporate a monitor program that will balance the loads and direct traffic to different servers to ensure that students are able to access the system. In addition, we can also incorporate caching to make data more available during peak times, such as when registration windows open.
  - As the registration process is so time-sensitive, ensuring availability is crucial. By having our system replicated across multiple servers, we can reduce the number of single points of failure. This will allow us to work towards the requirement of having the system readily available 99% of the time.

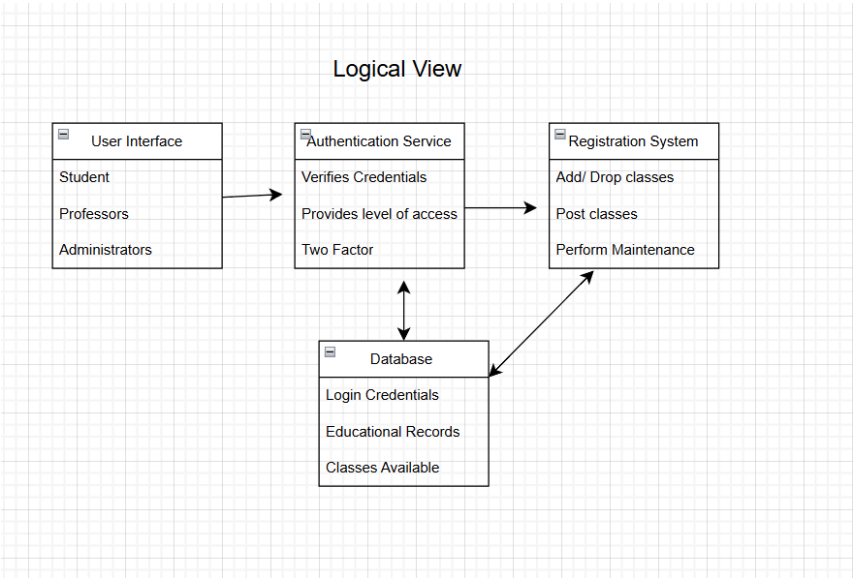
- The second non-functional requirement is that “the system shall only allow certain users to view students’ personal data as specified by FERPA.
  - This requirement is focused on the security and privacy of the students. The Privacy Act provides federal guidelines around a student's educational record that must be complied with.
  - To meet this requirement, we will implement multiple features for the system. First, there is a layered architecture, which is designed to make any personal information or students' data less accessible. For registration purposes, only the required information will be retrieved from the database, and only specific users will be able to access this information. In addition, we will authenticate students using a two-factor method to ensure that the individual registering is, in fact, the student themselves.

- Although this decision may reduce the performance due to taking longer to access data, this is a tradeoff that we cannot avoid. Not only because there are federal regulations around this data, but also to protect the users themselves. By ensuring that all of the private information is stored in a layer less accessible, we will achieve this requirement.

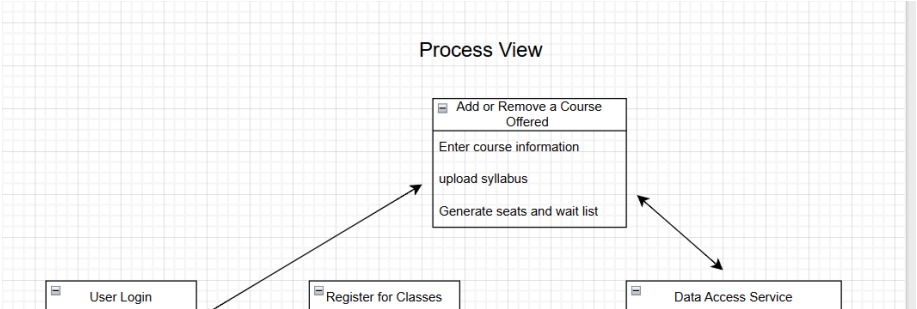
5. High-Level Architecture (4+1 Views)

- Use at least two views from the 4+1 model to represent your system's architecture:
  - **Logical View:** the main modules, classes, or components and their relationships.
  - **Process View:** how major processes, services, or subsystems interact at runtime.
- Provide a simple diagram for each and a short explanation (2-3 sentences) of what the view shows about your system.

Answer:



- The logical view shows how the classes and objects interact with each other. The user interface is used by all three roles, and will first go through the authentication service before eventually gaining access to the registration system. Both of these systems will also interact with the database to verify credentials, access components of the education records, and display classes for students to register.



- The process view focuses on runtime interactions. For our system, the core actions are students registering for classes, professors adding classes, and administrators performing maintenance or working through bugs. The login process validates the user and then allows them (depending on their access) to perform various other actions. For many of these, the action has to access data to display available classes or update the seats available once registration is complete.

6. Use of Architectural Patterns

- Identify one or two architectural patterns (e.g., Layered, Client-Server, MVC, Repository, Pipe-and-Filter) that best fit your system.
    - Explain why you chose these patterns.
    - Discuss one strength and one limitation of each pattern in the context of your system.

Answer:

- Layered: Our class registration system has different responsibilities that can be represented in layers. From UI→Verification→Business Logic→ DataBase. Moreover, it enforces a clean organization, making it easy to assign who works on what and identifying which layer needs fixing.
    - One benefit of using a layered architecture is the ease of updating or replacing a specific layer. We are able to implement new features or replace a whole layer, such as a new database or a UI refresh, without

affecting the whole system. This is important because during registration time, it is a high priority to handle bugs as fast as possible and without affecting the rest of the system, so students can register for classes.
    - One limitation of using this architecture is that it affects performance. Having a layered architecture means that each layer is dependent on the others, and different layers can't communicate directly without needing to go through the layers between them. Sometimes this causes requests to pass through unnecessary layers. This affects performance and sometimes limits the flexibility of the system. Such as when a student just wants to see available classes, that request needs to go through each layer.
- MVC: Using an MVC architecture suits our class registration system because, in practice, students interact with a UI, request class information, or when anybody logs in, the system verifies that person's credentials. We are able to separate system data(model), the user interface(view), and the request(controller).
    - One benefit is the ability of parallel development to develop different parts much more easily. Moreover, developers can be organized based on a single thing, such as frontend developers(UI), backend developers(Controllers and Models), and developers focused on databases.
    - One limitation of using this architecture is that the controller part is too complex. This is very true for our class registration system because the student alone needs a lot of requirements. Such as dropping a class, searching up a specific class, filtering by professor, viewing their current schedule, and signing up for a waitlist if the class is full. The more features we develop, the bigger the controller becomes.